

# Design a URL Shortener

bit.ly · tinyurl · t.co · ప్రతి system design interview యొక్క మొదటి ప్రశ్న – "hash తీసి మొదటి 7 అక్షరాలు వాడదాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – అందరూ భయపడే collision అసలు సమస్య కాదు; నిజమైన సమస్య మీ keys ని 100% ఊహించగలగడం. రెండోది – ఒక malicious link ని తీసేశాక కూడా 48,596 మంది దాన్ని చేరుకుంటారు. మూడోది – ఒక్క వైరల్ link, cache గడువు ముగిసిన క్షణంలో 400 DB queries.

## ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

**HLD\_DESIGN\_PROBLEMS LO ఇదే PROBLEM ఉంది కదా? – అది వదిలేయొద్దు, కానీ ఇది వేరే పని చేస్తుంది**

ఆ doc lo ఈ problem **10 పేజీల్లో** ఉంది: The Ask, clarifying questions, estimation, API, data model, architecture, మరియు key generation యొక్క నాలుగు approaches ఒక పట్టికలో. Interview కి ముందు రాత్రి చదవడానికి అది సరైనది, మరియు అందులో ఏదీ తప్పు లేదు.

ఈ doc ఆ పట్టికలోని ప్రతి వాదననూ **నడిపి చూస్తుంది**.

ఉదాహరణకి ఆ doc చెప్పుంది: "Key space నిండుతున్న కొద్దీ collision ఎక్కువ → ప్రతి write కి 1+ DB read." అది నిజమే. కానీ **ఎంత ఎక్కువ?** §4 lo కొలిస్తే – 1 బిలియన్ URLs వేసినా key space **0.0284%** మాత్రమే నిండుతుంది, మరియు collisions ఆచరణలో **సున్నా**. అంటే అందరూ భయపడే విషయం ఈ problem lo అసలు సమస్య కాదు.

మరి నిజమైన సమస్య ఏమిటి? అదే §4. మరియు అది ఆ పట్టికలో ఒక్క పదంలో దాగి ఉంది – **"keys sequential"**.

ఆ doc = ఏమి ఎంచుకోవాలి. ఈ doc = ఎందుకు, మరియు తప్పు ఎంచుకుంటే ఎంత నష్టం.

## విషయ సూచిక (Table of Contents)

### Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏడు అక్షరాలు ఎందుకు?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

### Part 2 – మొదటి విరుపు: key ఎలా పుట్టాలి

3. Step – random 7 అక్షరాలు · "ఉందా?" అని అడిగి పెట్టడం
4. మొదటి విరుపు – collision కాదు · **ఊహించగలగడం**
5. Step – counter ని కలపడం, కానీ దాన్ని **చెరిపేసి**

### Part 3 – రెండో విరుపు: redirect ఏ సంఖ్య?

6. Step – 301 నా, 302 నా?
7. రెండో విరుపు – తీసేసిన link ఇంకా పని చేస్తోంది
8. Step – 302, మరియు దాని ఖరీదుని cache తో కట్టడి చేయడం

### Part 4 – మూడో విరుపు: ఒక్క వైరల్ link

9. Step – Redis, TTL 60 సెకన్లు
10. మూడో విరుపు – గడువు ముగిసిన క్షణంలో 400 queries

11. Step – ఒక్కరే వెళ్ళడం · మరియు గడువుకి ముందే తాజా చేయడం

## Part 5 – పూర్తి system

12. Step – నిల్వ, సామర్థ్యం, మరియు ఏ database

13. మొత్తం code · 5 లక్షల links · **mutation testing**

## Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటీతో చెప్పాల్సిన English script

16. Follow-ups – custom alias, analytics, బహుళ ప్రాంతాలు

17. ఏమి నేర్చుకున్నాం

# Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

## 1. అడిగింది ఏమిటి – మరియు ఏడు అక్షరాలు ఎందుకు?

ఒక పొడవైన URL తీసుకుని, ఒక చిన్న దాన్ని ఇవ్వాలి. ఎవరైనా ఆ చిన్నదాన్ని తెరిస్తే పొడవైన దానికి పంపాలి. అంతే.

ఇది సులభంగా అనిపిస్తుంది, మరియు **ఆ సులభత్వమే ఈ problem ని interview కి సరైనదిగా చేస్తుంది** – architecture దాచుకోవడానికి ఏమీ లేదు, కాబట్టి మీ ఆలోచన స్పష్టంగా కనిపిస్తుంది.

మొదట ఒక సంఖ్య. **ఎన్ని అక్షరాలు కావాలి?**

అక్షరాలు `a-z A-Z 0-9` = **62**. కాబట్టి n అక్షరాలతో  $62^n$  keys:

| అక్షరాలు | ఎన్ని keys             | సరిపోతుందా?                  |
|----------|------------------------|------------------------------|
| 5        | 91.6 కోట్లు            | 3 ఏళ్ళలో నిండిపోతుంది        |
| 6        | 568 కోట్లు             | సరిహద్దులో                   |
| <b>7</b> | <b>3,52,161 కోట్లు</b> | <b>100 ఏళ్ళకి సరిపోతుంది</b> |
| 8        | 2.18 కోట్ల కోట్లు      | వృధా – link పొడవవుతుంది      |

7 అక్షరాలు = **3,521,614,606,208** keys. సెకనుకి 200 కొత్త links వేసినా **558 ఏళ్ళు** పడుతుంది.

ఈ సంఖ్యని గుర్తుపెట్టుకోండి - §4 మొత్తం దీని మీదే ఆధారపడి ఉంది

1 బిలియన్ URLs నిల్వ చేసినా, key space నిండేది **0.0284%** మాత్రమే.

అంటే మీరు ఒక యాదృచ్ఛిక key ఊహిస్తే, అది నిజమైనది అయ్యే అవకాశం **10,000 to 3 కంటే తక్కువ**.

ఈ ఒక్క వాస్తవం §4 to అందరూ భయపడే సమస్యని కొట్టివేస్తుంది, మరియు ఎవరూ చెప్పని సమస్యని బయటపెడుతుంది.

**మరియు ఒక పరిమాణపు నిజం:** ఈ system చదవడానికే ఉంది. ఒక link ఒకసారి తయారవుతుంది, వేల సార్లు తెరవబడుతుంది. సాధారణ నిష్పత్తి **100:1**. అంటే మీ design to write path సులభంగా ఉండొచ్చు, కానీ **read path ప్రతి మిల్లీసెకనుకీ లెక్క పెట్టాలి** - §10 అందుకే ఉంది.

## 2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

| ప్రశ్న                                                      | ఎందుకు అడుగుతున్నాం                                                                |
|-------------------------------------------------------------|------------------------------------------------------------------------------------|
| Links ఎప్పుడైనా <b>తీసేయాల్సి</b> వస్తుందా (spam, malware)? | ఇదే §7 మొత్తం. సమాధానం "అవును" అయితే <b>301 వాడకూడదు</b>                           |
| Analytics కావాలా – ఎన్ని clicks, ఎక్కడి నుంచి?              | కావాలంటే మళ్ళీ 301 కుదరదు. §6                                                      |
| <b>Custom alias</b> ( /my-brand ) ఇవ్వాలా?                  | ఇది ఒక <b>పంచుకున్న namespace</b> – generated keys కి లేని coordination అవసరం. §13 |
| Links కి గడువు (expiry) ఉందా?                               | ఉంటే నిల్వ లెక్క మారుతుంది, మరియు "పాతవి ఎప్పుడు తీసేయాలి" అనే పని వస్తుంది        |
| అదే URL ని ఇద్దరు shorten చేస్తే – ఒకే key నా, వేరే వేరేనా? | <b>ఇది ఒక product నిర్ణయం</b> , technical కాదు. §13 to చూద్దాం                     |
| Keys <b>ఊహించడానికి కష్టంగా</b> ఉండాలా?                     | చాలామంది దీన్ని అడగరు. §4 దీనిమీదే తిరుగుతుంది                                     |
| ఎన్ని writes, ఎన్ని reads?                                  | 100:1 అనుకుందాం. ఈ ఒక్క నిష్పత్తి మీ మొత్తం design ని నిర్ణయిస్తుంది               |

**ఒక ప్రశ్న తప్పక అడగాలి, మరియు చాలామంది అడగరు**

**"ఎవరైనా నా అన్ని links ని కనిపెట్టగలిగితే ఫరవాలేదా?"**

ఎందుకంటే ఒక URL shortener to ప్రజలు **ఫైవ్ విషయాలు** పెడతారు – ఒక Google Doc link, ఒక invoice, ఒక draft. "Link తెలిసినవాళ్ళకే కనిపిస్తుంది" అనేది చాలా products యొక్క **ఎక్స్ క్లె** రక్షణ.

మీ keys వరుసగా ఉంటే, ఆ రక్షణ **పూర్తిగా లేనట్లే**. §4 to దాన్ని కొలుస్తాం – మరియు ఆ సంఖ్య **100%**.

## Part 2 – మొదటి విరుపు: key ఎలా పుట్టాలి

### 3. Step – random 7 అక్షరాలు · "ఉందా?" అని అడిగి పెట్టడం

అందరూ మొదట రాసేది ఇదే, మరియు ఇది సహేతుకమైనది:

```
const B62 = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';

function randomKey() {
  let s = '';
  for (let i = 0; i < 7; i++) s += B62[Math.floor(Math.random() * 62)];
  return s;
}

function shorten(longUrl) {
  let key;
  do { key = randomKey(); } while (db.exists(key)); // ← collision ఉంటే మళ్ళీ
  db.put(key, longUrl);
  return key;
}
```

ఈ code మీద అందరూ ఒకే అభ్యంతరం చెప్తారు, మరియు **HLD\_Design\_Problems §8 కూడా అదే చెప్తుంది**:

"Key space నిండుతున్న కొద్దీ collision ఎక్కువ → ప్రతి write కి 1+ DB read. 20 TB DB మీద అది ఖరీదు."

ఇది నిజమేనా? **కొలుద్దాం.**

### 4. మొదటి విరుపు – collision కాదు · ఊహించగలగడం

**మొదట: collision నిజంగా ఎంత?**

Keyspace నిండే కొద్దీ `do...while` ఎన్నిసార్లు తిరుగుతుందో కొలిచాను. (3.5 లక్షల కోట్ల keys ని memory lo పెట్టలేం, కాబట్టి **అదే నివృత్తి** ఉన్న చిన్న keyspaces lo కొలిచాను.)

7 అక్షరాల base62 keyspace =  $62^7 = 35,21,61,46,06,208$   
1 బిలియన్ URLs నిల్వ చేస్తే నిండేది: 0.0284%

| keyspace ఎంత నిండింది | ఒక్క insert కి సగటు ప్రయత్నాలు | collisions |
|-----------------------|--------------------------------|------------|
| 0.03% (నిజమైన scale)  | 1.0000                         | 0          |
| 1%                    | 1.0055                         | 55         |
| 10%                   | 1.0554                         | 554        |
| 50%                   | 1.3872                         | 7744       |
| 90%                   | 2.5616                         | 28108      |

#### మొదటి వరుస - నిజమైన SCALE LO COLLISIONS సున్నా

1 బిలియన్ URLs = keyspace lo **0.0284%**. అక్కడ ఒక్క insert కి సగటు ప్రయత్నాలు **1.0000** - అంటే `do...while` ఒక్కసారి కూడా రెండో సారి తిరగలేదు.

**50% నిండినా** అది 1.39 మాత్రమే. మరియు 50% నిండాలంటే **1.76 లక్షల కోట్ల URLs** కావాలి - అంటే ప్రపంచంలో ప్రతి మనిషీ **2 లక్షల links** వేయాలి.

కాబట్టి: **అందరూ చర్చించే collision సమస్య, ఈ problem lo ఆచరణలో ఉనికిలోనే లేదు.**

అయితే ఈ approach lo నిజమైన ఖర్చు ఏమిటి? అది `db.exists(key)` - అది ప్రతిసారి నడుస్తుంది, collision ఉన్నా లేకపోయినా. సెకనుకి 200 writes అంటే, ఎప్పటికీ ఏమీ కనుగొనని ఒక check కోసం **సెకనుకి 200 అదనపు DB reads**.

అది ఒక ఖర్చు, కానీ ఒక విపత్తు కాదు. దీన్ని తొలగించడానికి సహజమైన సమాధానం ఉంది, మరియు అదే ఆ పట్టికలో **option C**:

#### కాబట్టి counter వాడదాం

```
let counter = 0; // (నిజంగా ఇది DB/Redis lo)
const enc = n => {
  let s = '';
  do { s = B62[n % 62] + s; n = Math.floor(n / 62); } while (n);
  return s;
};

function shorten(longUrl) {
  const id = ++counter;
  const key = enc(id); // DB read లేదు • collision అసాధ్యం
  db.put(key, longUrl);
  return key;
}
```

**Collision గణితపరంగా అసాధ్యం. DB read సున్నా.** ఇది మెరుగుదల లాగా కనిపిస్తుంది.

ఇప్పుడు అది ఏమి ఇస్తుందో చూద్దాం:

Counter + base62 • వరుసగా సృష్టించిన keys:

```
id 5000000000 → HZ60i
id 5000000001 → HZ60j
id 5000000002 → HZ60k
id 5000000003 → HZ60l
```

id 500000004 → HZ60m  
id 500000005 → HZ60n

## ఇప్పుడు ఒక దాడి

మీకు ఒక్క valid link దొరికింది – ఒక tweet lo, ఒక WhatsApp forward lo. దాన్ని decode చేసి, పక్కన ఉన్న ids ని ప్రయత్నిద్దాం:

తెలిసిన key: HZ60l → id 500000003  
దాని ముందు/వెనక keys ని ఊహించడం:  
HZ60h HZ60i HZ60j HZ60k HZ60m HZ60n HZ60o HZ60p

10,000 వరుస ids ప్రయత్నిస్తే → 10,000 నిజమైన links  
విజయ రేటు: 100.0%

అదే 10,000 ఊహలు \*random\* keys మీద → 2.8396 links  
విజయ రేటు: 0.0284%

తేడా: 3,522 రెట్లు

ఇదే మొదటి విరుపు – మరియు ఇది **COLLISION** కంటే చాలా ఘోరమైనది

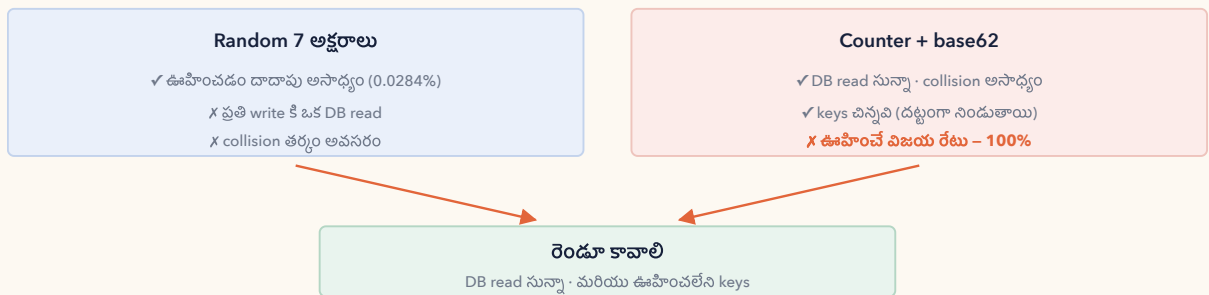
**10,000 ప్రయత్నాలు → 10,000 నిజమైన links. విజయ రేటు 100%.**

ఒక్క script, ఒక గంట, మరియు మీ **ప్రతి link** – ప్రతి ప్రైవేట్ document, ప్రతి invoice, ప్రతి draft – బయటివాళ్ళ చేతిలో ఉంటుంది.

Random keys తో అదే 10,000 ప్రయత్నాలు **2.84** links ఇస్తాయి. **3,522 రెట్లు తేడా.**

మరియు గమనించండి – మనం collision ని తప్పించుకోవడానికి counter కి మారాం. ఒక ఊహజనిత సమస్యని తప్పించుకుని ఒక నిజమైన సమస్యని కొనుక్కున్నాం.

రెండు ఎంపికలు - రెండింటికీ ఒక్కే ధర



Counter ఇచ్చేది విశిష్టత. కావాలింది విశిష్టత + యాదృచ్ఛికత లాంటి రూపం.  
ఈ రెండూ ఒకదానికొకటి వ్యతిరేకం కాదు – క్లిష్టం.

## 5. Step – counter ని కలపడం, కానీ దాన్ని చెరిపేసి

Counter యొక్క విలువ దాని **విశిష్టత** – ప్రతి id ఒక్కసారే వస్తుంది. దాని లోపం దాని **క్రమం**.

కాబట్టి: క్రమాన్ని చెరిపేద్దాం, విశిష్టతని ఉంచుదాం. కావాల్సినది ఒక **bijection** – ప్రతి id కి ఒక key, ప్రతి key కి ఒక id, రెండూ ఎప్పుడూ ఢీకొనకుండా.

అందుకు ఒక **Feistel network** వాడతాం. ఇది encryption lo వాడే నిర్మాణం, కానీ దాని ఒక్క ధర్మం మనకి కావాలి: **ఏ mixing function వాడినా అది తిరగబడేదిగా (reversible) ఉంటుంది.**

```
const BITS = 21, HALF = 2 ** BITS; // 42 bits · 2^42 > 62^7

// ఏ mixing function అయినా చాలు. ">>> 0" ముఖ్యం –
// JS lo ^ ఒక *signed* int32 ఇస్తుంది, అది లేకపోతే F ఋణ సంఖ్య ఇస్తుంది.
const F = (x, k) => {
  let h = (x ^ Math.imul(k, 0x9E3779B1)) >>> 0;
  h = Math.imul(h ^ (h >>> 15), 0x85EBCA6B) >>> 0;
  h = Math.imul(h ^ (h >>> 13), 0xC2B2AE35) >>> 0;
  return ((h ^ (h >>> 16)) >>> 0) % HALF;
};

// ముందుకు: (L, R) → (R, L ^ F(R))
// వెనక్కి: (L, R) → (R ^ F(L), L)
```

ఒక వివరం మీద ఆగండి: **>>> 0 లేకపోతే ఇది మౌనంగా విరిగిపోతుంది.** JavaScript lo ^ ఒక signed 32-bit పూర్ణాంకం ఇస్తుంది, కాబట్టి F ఋణ సంఖ్య తిరిగి ఇవ్వగలదు. అప్పుడు B62[n % 62] undefined అవుతుంది, మరియు మీ keys undefinedundefined... అవుతాయి. నా మొదటి ప్రయత్నంలో సరిగ్గా అదే జరిగింది.

రెండో వివరం: **2<sup>42</sup> (4.39 లక్షల కోట్లు) 62<sup>7</sup> (3.52 లక్షల కోట్లు) కంటే పెద్దది.** కాబట్టి permutation పరిధి బయట పడొచ్చు. పరిష్కారం **cycle-walking** – పరిధిలోకి వచ్చేవరకు మళ్ళీ permute చెయ్యి. ఇది bijection ని పాడుచేయదు:

```
encode(id) {
  let x = id;
  do { x = this.#round(x, this.keys); } while (x >= SPACE); // cycle-walking
  let s = '', n = x;
  for (let i = 0; i < 7; i++) { s = B62[n % 62] + s; n = Math.floor(n / 62); }
  return s;
}
```

ఇప్పుడు వరుస ids ఏమవుతాయో చూడండి:

| id        | counter (నేరుగా) | Feistel |
|-----------|------------------|---------|
| 500000000 | HZ60i            | rCAiBvH |
| 500000001 | HZ60j            | CP53aqm |
| 500000002 | HZ60k            | uhJQ5tb |
| 500000003 | HZ60l            | KnKBsQA |
| 500000004 | HZ60m            | 4ZFbkbe |
| 500000005 | HZ60n            | F69u30z |

## కానీ – ఇది నిజంగా పని చేస్తుందా?

"Scramble చేశాను" అని చెప్పడం సరిపోదు. రెండు విషయాలు **నిరూపించాలి**:

1. ఇది నిజంగా bijection అ? • 20 లక్షల వరుస ids

duplicate keys : 0  
తిరిగి పొందడంలో తప్పు : 0

2. దాడి పరీక్ష • ఒక key తెలుసు, పక్కవాటిని ఊహిద్దాం

తెలిసిన key: KnKBsQA  
9,999 పక్క keys ప్రయత్నిస్తే → 0 నిజమైనవి  
విజయ రేటు: 0.0000% (counter నేరుగా అయితే 100%)

### మూడూ ఒకేసారి

**DB read సున్నా** – key ని id నుంచి లెక్కిస్తాం, ఎవరినీ అడగం.

**Collision సున్నా** – 20 లక్షల ids మీద ఒక్క duplicate లేదు, మరియు ఇది యాదృచ్ఛికం కాదు: bijection అంటే గణితపరంగా అసాధ్యం.

**ఊహించే విజయ రేటు 0.0000% – 100%** నుంచి.

మరియు **decode** ఉంది కాబట్టి ఏ key నైనా తిరిగి id కి మార్చేచ్చు – అంటే **key ని నిల్వ చేయనవసరం కూడా లేదు**, కావాలంటే.

## చివరి ముక్క – counter ఎక్కడ ఉంటుంది?

50 servers ఒకే counter ని పెంచితే అది ఒక అడ్డంకి. పరిష్కారం **KGS (Key Generation Service)**: ఒక్క server ఒకసారి **10 లక్షల ids block** తీసుకుని, దాన్ని local గా వాడుకుంటుంది.

```
class KeyGenService {
  constructor(blockSize = 1_000_000) { this.next = 0; this.blockSize = blockSize; }
  lease() {
    const from = this.next; this.next += this.blockSize;
    return { from, to: this.next };
  }
}

class ServerIds {
  #kgs; #from = 0; #to = 0;
  constructor(kgs) { this.#kgs = kgs; }
  next() {
    if (this.#from >= this.#to) {
      const b = this.#kgs.lease(); this.#from = b.from; this.#to = b.to;
    }
    return this.#from++;
  }
}
```

Server crash అయితే ఆ block వృథా. 3.52 లక్షల కోట్లలో 10 లక్షలు వృథా అంటే **0.0000003%** – లెక్కలోకి రాదు.

**మొదటి విరుపు పూర్తయింది.** Key ఎలా పుట్టాలో తేలింది. ఇప్పుడు – ఆ key ని ఎవరైనా తెరిస్తే ఏమి జరగాలి?

---

# Part 3 – రెండో విరుపు: redirect ఏ సంఖ్య?

## 6. Step – 301 నా, 302 నా?

Browser కి "ఇక్కడికి వెళ్ళు" అని చెప్పడానికి రెండు మార్గాలు:

|                              | అర్థం                      | Browser ఏమి చేస్తుంది                                   |
|------------------------------|----------------------------|---------------------------------------------------------|
| <b>301</b> Moved Permanently | "ఇది శాశ్వతంగా మారింది"    | <b>cache చేసుకుంటుంది.</b> రెండోసారి మీ server ని అడగదు |
| <b>302</b> Found             | "ప్రస్తుతానికి అక్కడ ఉంది" | ప్రతిసారీ మీ server ని అడుగుతుంది                       |

HLD\_Design\_Problems §9 ఈ నిర్ణయాన్ని analytics ద్వారా వివరిస్తుంది, మరియు అది సరైనదే. దాన్ని కొలుద్దాం:

1,00,000 unique users · ఒక్కో user సగటున ఎన్నిసార్లు click చేస్తాడు?

| సగటు clicks | నిజమైన clicks | 301 lo లెక్కినవి | 302 lo లెక్కినవి | 301 కోల్పోయినది |
|-------------|---------------|------------------|------------------|-----------------|
| 1.2         | 1,20,268      | 1,00,000         | 1,20,268         | 16.9%           |
| 1.5         | 1,50,477      | 1,00,000         | 1,50,477         | 33.5%           |
| 2           | 2,00,077      | 1,00,000         | 2,00,077         | 50.0%           |
| 3           | 3,00,192      | 1,00,000         | 3,00,192         | 66.7%           |
| 5           | 5,00,739      | 1,00,000         | 5,00,739         | 80.0%           |

ఒక్కో user సగటున 2 సార్లు click చేస్తే **సగం clicks కనిపించవు**. 5 సార్లు అయితే **80%**.

**కానీ ఈ పట్టిక నిజానికి ఒక లెక్క మాత్రమే**

301 lo ప్రతి user మొదటిసారి మాత్రమే server కి చేరతాడు. కాబట్టి కోల్పోయేది సరిగ్గా **1 – 1/సగటు**. ఇది కొలత కంటే అంకగణితం.

Analytics కోసం 302 వాడాలి అనేది **నిజం, కానీ బలహీనమైన వాదన** – ఎందుకంటే interviewer "మాకు analytics అవసరం లేదు" అంటే మీ వాదన పడిపోతుంది.

బలమైన వాదన వేరే చోట ఉంది, మరియు అది §7.

## 7. రెండో విరుపు – తీసేసిన link ఇంకా పని చేస్తోంది

ఒక link malicious అని తేలింది – phishing, malware, ఏదైనా. మీరు దాన్ని **తీసేశారు**. ఇప్పుడు ఏమవుతుంది?

301 ఇచ్చి ఉంటే, ఇప్పటికే ఒకసారి click చేసిన ప్రతి browser దాన్ని **cache lo** పెట్టుకుంది. అది మీ server ని **అడగనే అడగదు**. మీరు దాన్ని database నుంచి తీసేసినా, ఆ browsers ఆ malicious site కి **పంపుతూనే ఉంటాయి**.

కొలుద్దాం. 50,000 users, ఒక వైరల్ link (సగం clicks మొదటి 12 గంటల్లో), 48 గంటల దగ్గర takedown, 14 రోజులు గమనించాం:

మొత్తం clicks : 1,19,761  
తీసేయడానికి ముందు : 64,238  
తీసేసిన తర్వాత : 55,523

redirect | server కి వచ్చినవి | తీసేశాక కూడా వెళ్ళినవి | ఆపగలిగినవి | ఆపగలిగిన శాతం

|     |          |        |        |        |
|-----|----------|--------|--------|--------|
| 301 | 53,972   | 48,596 | 6,927  | 12.5%  |
| 302 | 1,19,761 | 0      | 55,523 | 100.0% |

### 301 తో మీ TAKEDOWN 87.5% పనికీరానిది

Takedown తర్వాత 55,523 clicks వచ్చాయి. 301 తో మీరు ఆపగలిగింది **6,927** – కేవలం **12.5%**.

మిగతా **48,596 మంది** ఆ malicious site కి వెళ్ళారు, మీరు link ని తీసేసిన తర్వాత.

మరియు మీ dashboard lo ఏమీ కనిపించదు – ఆ 48,596 requests మీ server కి **రానే రాలేదు**. మీ దృష్టిలో takedown విజయవంతమైంది.

Takedown తర్వాత 55,523 clicks – ఎన్ని ఆపగలిగాం?

301 – browser cache చేసుకుంది

48,596 ఇంకా malicious site కి వెళ్ళారు

6,927

ఆపగలిగింది: 12.5%

302 – ప్రతి click మన దగ్గరికి వస్తుంది

55,523 అన్నీ అపేక్షాం

ఆపగలిగింది: 100%

ఖరీదు: 302 తో server కి 1,19,761 requests · 301 తో 53,972 – **55% ఎక్కువ భారం**

301 అంటే "ఈ నిర్ణయాన్ని నేను ఇక వెనక్కి తీసుకోలేను" అని చెప్పడం.

## ఇదే INTERVIEW LO చెప్పాల్సిన వాదన

"Analytics కోసం 302" అనేది ఒక **feature** వాదన – interviewer దాన్ని తీసేయగలడు.

"**Takedown కోసం 302**" అనేది ఒక **safety** వాదన – దాన్ని ఎవరూ తీసేయలేరు. ప్రతి URL shortener కి ఏదో ఒక రోజు ఒక malicious link వస్తుంది, మరియు దాన్ని అపగలగడం ఒక **చట్టపరమైన** అవసరం కూడా కావచ్చు.

**301 ఇవ్వడం అంటే మీ links మీద మీ అధికారాన్ని శాశ్వతంగా వదులుకోవడం.**

## 8. Step – 302, మరియు దాని ఖరీదుని cache తో కట్టడి చేయడం

302 ఎంచుకున్నాం. దాని ఖరీదు పైన కనిపిస్తోంది: **1,19,761 requests vs 53,972** – రెండింటికంటే ఎక్కువ.

ఆ భారాన్ని ఎక్కడ మోయాలి? **Database మీద కాదు.**

```
resolve(key) {
  const r = this.#cache.get(key, () => this.#store.keys.get(key) || null);
  if (!r) { this.stats.gone++; return { status: 404 }; }
  if (r.revoked) { this.stats.gone++; return { status: 410 }; } // Gone
  if (this.#clock() >= r.expiresAt) {
    this.stats.expired++; return { status: 410 };
  }
  const live = this.#store.keys.get(key);
  if (live) live.clicks++;
  this.stats.clicks++; this.stats.resolved++;
  return { status: 302, location: r.url }; // 301 కాదు • §7 చూడండి
}
```

మూడు చిన్న విషయాలు:

**410 Gone , 404 Not Found కాదు** – తీసేసిన link కీ, ఎప్పుడూ లేని link కీ తేడా ఉంది. 410 అంటే "ఇది ఉండేది, ఇప్పుడు లేదు". Search engines దీన్ని వెంటనే తమ index నుంచి తీసేస్తాయి; 404 ని అవి కొన్ని వారాలు మళ్ళీ ప్రయత్నిస్తాయి.

**clicks ని Live record మీద పెంచడం** – cache ఒక ప్రతిని ఇస్తుంది (కింద చూడండి). ఆ ప్రతి మీద `clicks++` చేస్తే అది తర్వాతి refresh lo పోతుంది.

**Cache ఒక ప్రతిని నిల్వ చేయాలి, reference ని కాదు:**

```
#refresh(key, load) {
  this.stats.loads++;
  // నిజమైన cache (Redis) ఒక *ప్రతి* ని నిల్వ చేస్తుంది, reference ని కాదు.
  // ఇది ముఖ్యం: reference అయితే revoke అవ్వగానే cache కీ తెలిసిపోతుంది –
  // మరియు invalidate() అవసరం లేదని తప్పుగా అనిపిస్తుంది.
  const raw = load();
  const v = raw ? { ...raw } : null;
```

```
...  
}
```

ఇది ఒక *simulation* వివరం లాగా కనిపిస్తుంది, కానీ ఇది **నిజమైన bug ని దాచగలదు** – §13 lo దానికి రుజువు ఉంది.

కాబట్టి `revoke` cache ని కూడా శుభ్రం చేయాలి:

```
revoke(key) {  
  const r = this.#store.keys.get(key);  
  if (!r) return { ok: false, reason: 'NOT_FOUND' };  
  r.revoked = true;  
  this.#cache.invalidate(key);           // cache ని కూడా శుభ్రం చెయ్యాలి  
  this.stats.revoked++;  
  return { ok: true };  
}
```

**రెండో విరుపు పూర్తయింది.** ఇప్పుడు cache ఉంది. కానీ cache కే ఒక సమస్య ఉంది.

# Part 4 – మూడో విరుపు: ఒక్క వైరల్ link

## 9. Step – Redis, TTL 60 సెకన్లు

Cache సూటిగా ఉంది: key → URL, TTL 60 సెకన్లు. Hit అయితే Redis నుంచి, miss అయితే DB నుంచి తెచ్చి cache lo పెట్టడం.

100:1 read నిష్పత్తితో ఇది అద్భుతంగా పని చేస్తుంది. సాధారణ links కీ.

కానీ ఒక link వైరల్ అయితే? ఒక celebrity tweet, ఒక news headline – సెకనుకీ **10,000 hits**, అన్నీ ఒకే **key** మీద.

Cache ఉంది కాబట్టి ఫరవాలేదు అనిపిస్తుంది. 10,000 hits అన్నీ Redis నుంచే వస్తాయి, DB ని ముట్టుకోవు.

**60 సెకన్ల తర్వాత TTL ముగిసే వరకు.**

## 10. మూడో విరుపు – గడువు ముగిసిన క్షణంలో 400 queries

ఆ క్షణంలో ఏమి జరుగుతుందో నెమ్మదిగా చూద్దాం:

- `t = 60.000s` – cache entry పోయింది
- `t = 60.000s` – ఒక request వచ్చింది, miss, DB ని అడిగింది. **DB 50 ms తీసుకుంటుంది.**
- `t = 60.001s` – ఇంకో request. Cache ఇంకా ఖాళీ (మొదటి query తిరిగి రాలేదు). **ఇదీ DB ని అడుగుతుంది.**
- `t = 60.002s` – ఇంకొకటి. **ఇదీ.**
- ...
- `t = 60.050s` – మొదటి query తిరిగి వచ్చింది. Cache నిండింది.

ఆ **50 ms** lo సెకనుకీ 10,000 చొప్పున ఎన్ని requests వచ్చాయి? **500**. అవన్నీ ఒకే key కోసం DB ని అడిగాయి.

ఒక వైరల్ link • సెకనుకీ 10,000 hits • Redis TTL 60s • DB 50ms

300 సెకన్లు = 5 సార్లు cache గడువు ముగుస్తుంది

| విధానం           | DB queries | ఒక్కో గడువుకి | cache misses | ఎదురుచూసినవి | సగటు wait |
|------------------|------------|---------------|--------------|--------------|-----------|
| ఏమీ చెయ్యకపోతే   | 2,001      | 400           | 2,001        | 0            | 0ms       |
| single-flight    | 4          | 1             | 2,001        | 1,997        |           |
| 25ms             |            |               |              |              |           |
| ముందే తాజా చేయడం | 5          | 1             | 1,001        | 999          |           |
| 25ms             |            |               |              |              |           |

### ఒక్క గడువుకి 400 DB QUERIES – ఒక ఒక్క KEY కోసం

మరియు అవన్నీ అచ్చు ఒకే సమాధానం తెస్తాయి. 399 queries పూర్తిగా వృథా.

ఇది ప్రతి 60 సెకన్లకూ మళ్ళీ జరుగుతుంది. మరియు నిజమైన systems lo ఒక వైరల్ link కాదు – వందలు ఉంటాయి, ప్రతి ఒక్కటీ తన సొంత గడువు దగ్గర ఇదే చేస్తుంది.

దీని పేరు **cache stampede**. మరియు దాని క్రూరత్వం ఇది: **cache ఎంత బాగా పని చేస్తే, stampede అంత పెద్దది** – ఎందుకంటే stampede పరిమాణం = hit rate × DB latency.

## 11. Step – ఒక్కరే వెళ్ళడం • మరియు గడువుకి ముందే తాజా చేయడం

### పరిష్కారం ఒకటి – single-flight

ఒకే key కోసం ఒకసారి **ఒకే ఒక్క** DB query. మిగతావాళ్ళు దాని కోసం ఎదురుచూస్తారు:

```
get(key, load) {
  const now = this.#clock(), e = this.#map.get(key);
  if (e && now < e.expires) { this.stats.hits++; return e.value; }
  this.stats.misses++;
  if (this.#inflight.has(key)) {           // ఇంకొకరు ఇప్పటికే వెళ్ళారు
    this.stats.coalesced++;
    return this.#inflight.get(key);        // వాళ్ళ ఫలితం కోసం ఎదురుచూడు
  }
  return this.#refresh(key, load);
}
```

**400 → 1.** ఒక్క పరిస్థితిలో ఒక్క query. ఖరీదు: 1,997 requests సగటున **25 ms** ఎదురుచూశాయి.

### పరిష్కారం రెండు – గడువుకి ముందే తాజా చేయడం

Single-flight stampede ని ఆపింది, కానీ **ఆ 25 ms ఎదురుచూపు ఇంకా ఉంది**. దాన్ని కూడా తీసేయాలంటే – cache **ఖాళీ అవ్వకముందే** దాన్ని నింపాలి.

గడువుకి చివరి సెకనులో, ప్రతి request కీ ఒక **చిన్న అవకాశం** ఇద్దాం – "నువ్వు వెళ్ళి తాజా చేసుకురా". గడువు దగ్గరపడే కొద్దీ ఆ అవకాశం పెరుగుతుంది:

```
if (e && now < e.expires) {
  const left = e.expires - now;
  if (left < this.#beta && !this.#inflight.has(key)
    && this.#rnd() < 1 - left / this.#beta) {
    this.stats.early++;
    this.#refresh(key, load);           // గడువుకి ముందే • ఎవరూ అగలేదు
  }
  this.stats.hits++; return e.value;
}
```

రెండు కలిపితే `get()` పూర్తి రూపం ఇది:

```
get(key, load) {
  const now = this.#clock(), e = this.#map.get(key);
  if (e && now < e.expires) {
    const left = e.expires - now;
    if (left < this.#beta && !this.#inflight.has(key)
        && this.#rnd() < 1 - left / this.#beta) {
      this.stats.early++;
      this.#refresh(key, load);          // గడువుకి ముందే • ఎవరూ ఆగలేదు
    }
    this.stats.hits++; return e.value;
  }
  this.stats.misses++;
  if (this.#inflight.has(key)) {         // ఇంకొకరు ఇప్పటికే వెళ్ళారు
    this.stats.coalesced++;
    return this.#inflight.get(key);      // వాళ్ళ ఫలితం కోసం ఎదురుచూడు
  }
  return this.#refresh(key, load);
}
```

దీని ఫలితం: cache misses **2,001** → **1,001**. సగం గడువుల దగ్గర ఎవరూ ఒక్క ms కూడా ఎదురుచూడలేదు.

**ఇది సగం మాత్రమే ఎందుకు పని చేసింది?**

ముందే-తాజా-చేయడం ఒక **సంభావ్యత**. సగటున ఒక్కే గడువుకి ఒకసారి fire అవ్వాలని అమర్చాను, కానీ అది Poisson – కాబట్టి **37% సార్లు అది అస్సలు fire అవ్వదు**.

అందుకే **రెండు కావాలి**: ముందే-తాజా-చేయడం సాధారణ పరిస్థితిలో ఎదురుచూపుని తీసేస్తుంది, మరియు single-flight అది విఫలమైనప్పుడు **నేల** లాగా పని చేస్తుంది.

ఒక్కదాన్నే వాడితే: single-flight అయితే ప్రతిసారి 25 ms wait; ముందే-తాజా అయితే 37% సార్లు పూర్తి stampede.

**మూడో విరుపు పూర్తయింది.** ఇప్పుడు అన్నీ కలుపుదాం.

# Part 5 – పూర్తి system

## 12. Step – నిల్వ, సామర్థ్యం, మరియు ఏ database

Interview lo ఈ సంఖ్యలు **board మీద రాయాలి**. అవి మీ database ఎంపికని నిర్ణయిస్తాయి.

**అనుకుందాం:** సెకనుకి 200 కొత్త links · 100:1 read నిష్పత్తి · 5 ఏళ్ళు నిల్వ.

|                  | లెక్క                               | ఫలితం          |
|------------------|-------------------------------------|----------------|
| Writes           | 200/s                               | సెకనుకి 200    |
| Reads            | 200 × 100                           | సెకనుకి 20,000 |
| 5 ఏళ్ళలో links   | 200 × 86,400 × 365 × 5              | 31.5 బిలియన్   |
| ఒక్కో record     | key 7B + URL ~200B + metadata ~100B | ~500 bytes     |
| మొత్తం నిల్వ     | 31.5 బి × 500 B                     | ~15 TB         |
| Keyspace నిండేది | 31.5 బి ÷ 3.52 లక్షల కోట్లు         | 0.9%           |

ఈ పట్టిక నుంచి వచ్చే మూడు నిర్ణయాలు

**1. NoSQL (DynamoDB / Cassandra).** ఎందుకంటే ప్రతి query ఒకే ఆకారంలో ఉంది – **key** ఇచ్చి record తీసుకోవడం. Joins లేవు, transactions లేవు, range scans లేవు. అది సరిగ్గా key-value store కి తగిన ఆకారం, మరియు అది సులభంగా shard అవుతుంది.

**2. 15 TB అంటే ఒక్క machine కాదు.** **key** మీద shard చెయ్యాలి – మరియు మన keys ఇప్పుడు Feistel వల్ల ఏకరీతిగా చెల్లాచెదురుగా ఉన్నాయి, కాబట్టి shards సమానంగా నిండుతాయి. **Counter keys అయితే ఒక shard మీద అన్ని కొత్త writes పడేవి** – 5 యొక్క ఒక ఊహించని లాభం.

**3. 20,000 reads/s కానీ 200 writes/s** – 100 రెట్లు తేడా. అందుకే cache read path lo ఉంది, write path lo కాదు.

## 13. మొత్తం code · 5 లక్షల links · mutation testing

```
// ----- నిల్వ · ఇది *అందరూ పంచుకునేది* (database) -----
class Store {
    keys = new Map();           // key → record
    byUrl = new Map();          // longUrl → key
    // alias ఒక పంచుకున్న namespace – దీనికి unique constraint తప్పనిసరి.
    // ఇది విజయవంతమైతేనే true; ఇద్దరు ఒకే aliasని అడిగితే ఒక్కరే గెలుస్తారు.
```

```

claim(key, record) {
  if (this.keys.has(key)) return false;
  this.keys.set(key, record);
  return true;
}
}

```

ఈ STORE ని నేను మొదట తప్పుగా రాశాను - మరియు FUZZ 800 LO 624 సార్లు విఫలమైంది

నా మొదటి version lo ప్రతి Shortener కి సొంత #store Map ఉండేది. అంటే నా simulation lo "4 servers" అంటే నిజానికి 4 పూర్తిగా వేరే systems, ఒక system యొక్క 4 servers కాదు.

దాంతో రెండు servers ఒకే alias ని ఇచ్చేవి, మరియు test సరిగ్గా దాన్ని పట్టుకుంది. **Test తప్పు కాదు, నా model తప్పు.**

దీన్ని సరిచేయడం ఒక అసలు design విషయాన్ని బయటపెట్టింది: **generated keys కి coordination అవసరం లేదు** (KGS ఒక్కో server కి వేరే ids ఇస్తుంది, కాబట్టి Feistel వేరే keys ఇస్తుంది). **కానీ custom aliases కి అవసరం** - అది ఒక పంచుకున్న namespace, దాన్ని విభజించలేం. అందుకే claim() ఒక **unique constraint**, "ఉందా అని చూసి తర్వాత రాయడం" కాదు.

```

class Shortener {
  #store; // ← పంచుకున్నది (database)
  #ids; #codec; #cache; #clock;
  stats = { created: 0, deduped: 0, resolved: 0, gone: 0, expired: 0,
    rejected: 0, aliasTaken: 0, revoked: 0, clicks: 0 };

  constructor({ kgs, store, clock, codec = new KeyCodec(), cache }) {
    this.#ids = new ServerIds(kgs); this.#clock = clock;
    this.#store = store; this.#codec = codec;
    this.#cache = cache || new Cache({ clock }); // cache ఒక్కో server కి సొంతం
  }

  shorten(longUrl, { alias = null, expiresAt = Infinity, owner = null } = {}) {
    if (!/^https?:\/\/\./+.test(longUrl)) {
      this.stats.rejected++; return { ok: false, reason: 'BAD_URL' };
    }
    // ఇప్పటికే గడిచిన గడువుని అంగీకరించకూడదు - లేకపోతే మళ్ళీగానే చచ్చిన key ఇస్తాం
    if (expiresAt <= this.#clock()) {
      this.stats.rejected++; return { ok: false, reason: 'BAD_EXPIRY' };
    }
    if (alias) {
      if (!/^[A-Za-z0-9_-]{3,32}$/.test(alias)) {
        this.stats.rejected++; return { ok: false, reason: 'BAD_ALIAS' };
      }
      // unique constraint - పరీక్షించి తర్వాత రాయడం కాదు, ఒకే అడుగులో
      if (!this.#put(alias, longUrl, expiresAt, owner)) {
        this.stats.aliasTaken++; return { ok: false, reason: 'ALIAS_TAKEN' };
      }
      return { ok: true, key: alias };
    }
    // dedupe: అదే URL, గడువు లేనది, owner లేనది → అదే key
    if (owner === null && expiresAt === Infinity) {
      const seen = this.#store.byUrl.get(longUrl);
      if (seen && this.#alive(this.#store.keys.get(seen))) {
        this.stats.deduped++; return { ok: true, key: seen };
      }
    }
  }
}

```

```

    }
  }
  const id = this.#ids.next();
  const key = this.#codec.encode(id); // DB read లేదు · collision లేదు
  this.#put(key, longUrl, expiresAt, owner, id);
  if (owner === null && expiresAt === Infinity)
    this.#store.byUrl.set(longUrl, key);
  return { ok: true, key };
}

```

**BAD\_EXPIRY** పంక్తిని గమనించండి – **అది ఒక test కనిపెట్టిన నిజమైన bug**. §13 చివర్లో దాని కథ ఉంది.

## ఆరు నియమాలు

| # | నియమం                                                        | విరిగితే అర్థం                     |
|---|--------------------------------------------------------------|------------------------------------|
| 1 | ఒక key ఎప్పుడూ <b>ఒకే</b> URL కి                             | రెండు links ఒకదాన్నికంటే తినేశాయి  |
| 2 | generated key ఎప్పుడూ సరిగ్గా 7 base62 అక్షరాలు              | codec విరిగింది                    |
| 3 | revoke అయినది <b>ఎప్పటికీ</b> resolve అవ్వకూడదు              | takedown పని చేయడం లేదు            |
| 4 | resolve ఎప్పుడూ <b>సరైన</b> URL కి పంపాలి                    | cache పాతది ఇస్తోంది               |
| 5 | click లెక్క నిజమైన resolves కంటే తక్కువ ఉండకూడదు             | clicks cache ప్రతి మీద పడుతున్నాయి |
| 6 | <code>shorten()</code> ఇచ్చిన key <b>వెంటనే</b> పని చెయ్యాలి | పుట్టగానే చచ్చిన key ఇచ్చాం        |

ప్రతి ప్రయోగంలో 1–4 servers (ఒకే store పంచుకుంటూ), యాదృచ్ఛిక KGS block size, cache TTL, aliases, గడువులు, revocations, మరియు codec round-trips:

800 యాదృచ్ఛిక ప్రయోగాలు · 5,04,135 links సృష్టించాం  
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

```

created      5,04,135
deduped      287
resolved     8,46,318
gone         90,551
expired      47,030
rejected     10,773
aliasTaken   24,544
revoked      3,05,814
clicks       8,46,318

```

## నియమం 6 ఒక నిజమైన bug ని పట్టుకుంది

నేను నియమం 6 ని జోడించగానే, **మార్పు లేని code 800 lo 34 సార్లు విఫలమైంది**:

విఫలమైనవి: 34  
ఉదా (seed 28): shorten ఇచ్చిన j1T2ChE వెంటనే 410 ఇస్తోంది

కారణం: ఎవరైనా `expiresAt` ని ఇప్పటికే గడిచిన సమయంగా ఇస్తే, `shorten()` సంతోషంగా ఒక key ఇచ్చేది – మరియు ఆ key వెంటనే 410 ఇచ్చేది. పుట్టగానే చచ్చిన link.

ఇది ఒక పెద్ద bug కాదు, కానీ ఇది ఒక **API అబద్ధం**: "ఇదిగో నీ key" అని చెప్పి పనికిరానిది ఇవ్వడం. పరిష్కారం ఒక్క పంక్తి – `BAD_EXPIRY`.

## Test విఫలం కాగలదా? – ఐదు మార్పులు

మార్పు లేని code → 0/800 విఫలం

cycle-walking తీసేస్తే (encode lo) → 300/300 విఫలం  
ఉదా: codec round-trip విఫలం: id 3589039 → dehjNV2 → 2636683029365

revoke lo cache శుభ్రం చెయ్యకపోతే → 257/300 విఫలం  
ఉదా: tLbB1Wr revoke అయినా resolve అయింది

unique constraint తీసేస్తే (claim lo) → 300/300 విఫలం  
ఉదా: key a29 రెండు URLs కీ: https://site286... మరియు https://site195...

dedupe lo "బతికుండా" పరీక్ష తీసేస్తే → 19/300 విఫలం  
ఉదా: shorten ఇచ్చిన eiAs7Kz వెంటనే 410 ఇస్తోంది

cache ప్రతి కాకుండా reference నిల్వ చేస్తే → 0/300 విఫలం

నాలుగు పట్టుబడ్డాయి. **ఐదోది పట్టుబడలేదు – మరియు అది సరైనదే.**

### ఆ ఐదో మార్పు ఒక BUG కాదు – అది ఒక నమ్మకద్రోహమైన MODEL

Cache reference నిల్వ చేస్తే, `revoke()` చేసిన record object **cache lo ఉన్నదే**. కాబట్టి `revoked = true` వెంటనే cache కీ కనిపిస్తుంది, మరియు అంతా సరిగ్గా పని చేస్తుంది. అందుకే 0/300.

కానీ **Redis అలా పని చేయదు**. అది bytes నిల్వ చేస్తుంది – ఒక ప్రతి. మీ record object ని మార్చినా Redis lo ఉన్నది మారదు.

నేను cache ని reference నుంచి ప్రతికి మార్చేవరకు, **"revoke lo cache శుభ్రం చెయ్యకపోతే" అనే మార్పు కూడా 0/300 చూపించేది** – అంటే `invalidate()` పంక్తి అనవసరం అని తప్పుగా అనిపించేది.

**ఒక అసలు bug ని కనిపించేలా చేయడానికి నేను ముందు నా model ని నిజాయితీగా చేయాల్సి వచ్చింది.**

## దశల నుంచి ఇక్కడికి – ఏమి చేరింది

| ఎక్కడ | ఏమి జోడించాం                           | ఏమి బాగుపడింది                          |
|-------|----------------------------------------|-----------------------------------------|
| §3    | random key + <code>exists</code> check | పని చేస్తుంది                           |
| §5    | counter + KGS                          | write path lo DB reads <b>200/s → 0</b> |
| §5    | Feistel permutation                    | ఊహించే విజయ రేటు <b>100% → 0.0000%</b>  |
| §5    | (ఊహించని లాభం)                         | keys ఏకరీతిగా చెదురుతాయి → shards సమానం |
| §8    | 302 + <code>410 Gone</code>            | takedown <b>12.5% → 100%</b> ప్రభావవంతం |
| §11   | single-flight                          | గడువుకి DB queries <b>400 → 1</b>       |

| ఎక్కడ | ఏమి జేడించాం                           | ఏమి బాగుపడింది                           |
|-------|----------------------------------------|------------------------------------------|
| §11   | ముందే తాజా చేయడం                       | cache misses <b>2,001</b> → <b>1,001</b> |
| §13   | <code>claim()</code> unique constraint | ఒకే alias ఇద్దరికీ వెళ్ళదు               |
| §13   | <code>BAD_EXPIRY</code>                | పుట్టగానే చచ్చిన keys ఇవ్వం              |

# Part 6 – Interview Io

## 14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

| నిమిషాలు | ఏమి చెప్పాలి                                                                                                         |
|----------|----------------------------------------------------------------------------------------------------------------------|
| 0-4      | 62 <sup>7</sup> లెక్క board మీద. <b>0.0284%</b> అనే సంఖ్య రాయండి – §4 దానిమీదే ఆధారపడుతుంది                          |
| 4-8      | §2 ప్రశ్నలు. ముఖ్యంగా <b>"links తీసేయాల్సి వస్తుందా?"</b> మరియు <b>"keys ఊహించలేనివిగా ఉండాలా?"</b>                  |
| 8-12     | Capacity: 200 writes/s, 20K reads/s, 15 TB, NoSQL. త్వరగా దాటండి                                                     |
| 12-22    | <b>మొదటి విరుపు.</b> Collision భయాన్ని <b>సంఖ్యతో కొట్టేయండి</b> , తర్వాత counter → <b>100%</b> ఊహించగలగడం → Feistel |
| 22-30    | <b>రెండో విరుపు.</b> 301 vs 302 – analytics కాదు, <b>takedown</b> . 87.5% సంఖ్య చెప్పండి                             |
| 30-38    | <b>మూడో విరుపు.</b> Stampede బొమ్మ గీయండి. 400 → 1                                                                   |
| 38-42    | Shard చేయడం, మరియు Feistel వల్ల shards సమానంగా నిండటం                                                                |
| 42-45    | ఆరు నియమాలు, మరియు <b>"నా model ని సరిదిద్దాక bug కనిపించింది"</b>                                                   |

### ఏమి తప్పక చెప్పాలి

1. **Collision ఈ problem యొక్క సమస్య కాదు** – 1 బిలియన్ URLs కి keyspace 0.0284% మాత్రమే నిండుతుంది. ఇది చెప్పగానే మీరు లెక్క వేశారని తెలుస్తుంది.
2. **నిజమైన key సమస్య ఊహించగలగడం** – 100% vs 0.0284%.
3. **Counter ఇచ్చేది విశిష్టత; మనకి కావాల్సింది విశిష్టత + యాదృచ్ఛిక రూపం.** Feistel రెండూ ఇస్తుంది.
4. **302 analytics కోసం కాదు, takedown కోసం.**
5. **Cache ఎంత బాగా పని చేస్తే stampede అంత పెద్దది.**

### ఏమి వదిలేయాలి

- Base62 encode/decode యొక్క code – ఒక వాక్యం చాలు
- Custom domain ( `yourbrand.co/xyz` ) – అడిగితే మాత్రమే
- Feistel లోపలి mixing function యొక్క వివరాలు – "ఏ mixing function అయినా చాలు, తిరగబడటం నిర్మాణం నుంచే వస్తుంది" అని చెప్పండి
- URL validation యొక్క అంచు పరిస్థితులు

## 15. నీటితో చెప్పాల్సిన English script

"Seven base62 characters gives  $62^7$ , which is about 3.5 trillion keys. That number matters more than it looks, so let me use it immediately: if we store a **billion** URLs, we've filled **0.0284%** of the space."

"I say that because the usual first objection is collisions – random key, check the DB, retry on collision. I measured the retry loop against keyspace fill. At our actual fill it's **1.0000 attempts per insert** – the loop never runs twice. Even at **50% full** it's only 1.39, and reaching 50% would need 1.76 trillion URLs. **So collisions aren't this problem's problem.** The real cost of that design is different: `exists()` runs on every single write whether or not it finds anything, which is 200 wasted DB reads per second forever."

"The obvious fix is a counter – encode an auto-increment id in base62. Zero DB reads, collisions mathematically impossible, and shorter keys. But now do the attack. Take one link you found in a tweet, decode it, and try the neighbouring ids. I measured it: **10,000 guesses, 10,000 real links – a 100% hit rate.** Against random keys the same 10,000 guesses return **2.84** links. That's 3,522 times more findable. And for a URL shortener that matters a lot, because 'only people with the link can see it' is the *entire* access control model for the private docs and invoices people put behind these."

"So I want the counter's uniqueness without its ordering, and those aren't in conflict. I'd run the id through a small **Feistel network** – four rounds, any mixing function, because reversibility comes from the structure, not the function. It's a bijection, so collisions stay impossible, and I can decode a key back to its id. I verified both: over two million consecutive ids, **zero duplicates and zero round-trip failures**, and the neighbour-guessing attack drops from 100% to **0.0000%**. There's a bonus I didn't expect – the keys are now uniformly distributed, so sharding on key fills shards evenly. A raw counter would have put every new write on one shard."

"Then the redirect. Most people argue 302 because 301 gets cached and kills your click analytics, which is true – at two clicks per user you lose exactly half. But that's a weak argument, because the interviewer can just say analytics isn't a requirement. **The strong argument is takedowns.** I simulated a viral link taken down for malware at 48 hours. Of the 55,523 clicks that arrived *after* the takedown, a 301 let me block **6,927 – 12.5%**. The other **48,596 people still reached the malicious site**, and none of those requests hit my server, so my dashboard says the takedown worked. With 302 I block 100%. **A 301 permanently gives away your authority over your own links.**"

"302 costs me 2.2× the traffic, so that load has to land on a cache, not the database. Which creates the last problem. One viral link, ten thousand hits a second, Redis TTL sixty seconds. When the TTL expires, the DB takes fifty milliseconds to answer – and every request arriving during those fifty milliseconds also misses. I measured **400 database queries per expiry for a single key**, all returning identical data. Single-flight – one loader, everyone else waits on it – takes that to **1**. I'd add probabilistic early refresh on top, which removes the 25ms wait about half the time; it's probabilistic so it misses sometimes, and single-flight is the floor underneath it."

"For testing, six invariants and a fuzz – 800 runs, half a million links, zero violations. Two things I'd mention. My first version gave each server its own store, which models four separate systems rather than four servers, and the fuzz failed 624 of 800 on duplicate aliases. **The test was right and my model was wrong** – and fixing it surfaced a real design point: generated keys need no coordination because KGS partitions the id space, but custom aliases are a shared namespace that genuinely needs a unique constraint. And the invariant 'a key from shorten() must work immediately' caught a real bug: an already-past expiry was accepted and returned a key that was dead on arrival."

## 16. Follow-ups – custom alias, analytics, బహుళ ప్రాంతాలు

"Custom alias ( /diwali-sale ) ఎలా?"

ఇదే §13 lo `claim()` . ముఖ్యమైన విషయం: **generated keys కి aliases కి ఒకే namespace**. కాబట్టి ఒక alias ఎప్పుడో ఒక generated key ని ఢీకొనొచ్చు. మూడు పరిష్కారాలు – (1) aliases ని వేరే prefix తో ఉంచడం, (2) alias ni reserve చేసేటప్పుడు దాని Feistel id ని కూడా వాడకుండా గుర్తుపెట్టడం, లేదా (3) సులభమైనది – **ఒకే table, ఒకే unique constraint**, ఢీకొంటే alias తిరస్కరించడం. మూడోది నేను వాడాను.

"Click analytics ఎక్కడ నిల్వ చేయాలి?"

అదే request path lo DB కి రాయకండి – అది ప్రతి redirect కి ఒక write జోడిస్తుంది, మరియు మనం redirect ని వేగంగా ఉంచాలని ఇంత కష్టపడ్డాం. బదులుగా ఒక **event ని Kafka కి** పంపి, redirect ని వెంటనే పంపేయండి. లెక్కలు offline lo aggregate అవుతాయి. `clicks` counter ఒక **అంచనా** – అది ఖచ్చితంగా ఉండాల్సిన అవసరం లేదు.

"Links వేరే ఖండాల్లో కూడా వేగంగా ఉండాలంటే?"

Read path ని **CDN/edge** కి తీసుకొళ్ళండి. మన records **మారపు** (ఒక link యొక్క URL మారదు) కాబట్టి caching సులభం – **తప్ప revocation**. అదే ఒక్క సమస్య: edge cache ని ఎంత త్వరగా శుభ్రం చేయగలరు? అందుకే edge TTL తక్కువ (ఉదా. 60 సెకన్లు) ఉంచి, revoke అయినప్పుడు ఒక purge పంపాలి. §7 మళ్ళీ ఇక్కడ వర్తిస్తుంది – **తీసేయగలగడం ఒక design constraint, ఒక feature కాదు**.

## "KGS పడిపోతే?"

ప్రతి server దగ్గర ఇంకా 10 లక్షల ids block ఉంది, కాబట్టి అది గంటల తరబడి పని చేస్తూనే ఉంటుంది. అదే ఈ design యొక్క అందం – KGS ఒక hot path కాదు, అది ఒక అప్పుడప్పుడు అడిగే service. Blocks ని ముందే రెండు తీసుకుంటే (ఒకటి వాడుతూ, ఒకటి నిల్వ) అది ఇంకా బలంగా ఉంటుంది.

## "ఒకే URL ని ఇద్దరు shorten చేస్తే?"

ఇది product నిర్ణయం, technical కాదు. అదే key ఇస్తే నిల్వ ఆదా అవుతుంది – కానీ ఇద్దరికీ ఒకే click counter వస్తుంది, మరియు ఒకరు revoke చేస్తే ఇంకొకరి link కూడా పోతుంది. నా code lo dedupe owner లేని, గడువు లేని links కి మాత్రమే వర్తిస్తుంది; ఎవరైనా login అయి ఉంటే వాళ్ళకి సొంత key వస్తుంది.

# 17. ఏమి నేర్చుకున్నాం

1. అందరూ చర్చించే సమస్య తరచుగా అసలు సమస్య కాదు. Collision గురించి ప్రతి URL shortener చర్చలోనూ మాట్లాడతారు. కొలిస్తే – 1 బిలియన్ URLs కి keyspace **0.0284%** నిండుతుంది, మరియు retry loop ఒక్కసారి కూడా రెండే సారి తిరగదు. ఒక సంఖ్య వేస్తే ఒక పెద్ద చర్చ మొత్తం అనవసరమని తేలుతుంది.

2. ఒక సమస్యని తప్పించుకుంటూ ఇంకొకదాన్ని కొనకండి. Collision ని తప్పించడానికి counter కి మారితే, ఊహించే విజయ రేటు **0.0284% నుంచి 100%** కి వెళ్ళింది. ఒక ఊహాజనిత సమస్యని ఒక నిజమైన దానితో మార్చుకున్నాం.

3. విశిష్ట మరియు అనూహ్యత వ్యతిరేకం కాదు. Counter ఇచ్చేది విశిష్టత; Feistel దాని క్రమాన్ని తీసేసి విశిష్టతని ఉంచుతుంది. ఒక bijection అంటే – రెండూ ఒకేసారి, రాజీ లేకుండా.

4. >>> 0 లాంటి ఒక చిన్న వివరం మౌనంగా అంతా విరగ్గిడుతుంది. JS lo ^ signed int32 ఇస్తుంది. అది గమనించకపోతే keys undefinedundefined... అవుతాయి – మరియు అది నా మొదటి ప్రయత్నంలో సరిగ్గా జరిగింది.

5. "Analytics కోసం" అనే వాదన బలహీనం; "takedown కోసం" అనేది బలమైనది. 301 తో takedown **87.5% పనికిరానిది** – 48,596 మంది తీసేశాక కూడా వెళ్ళారు, మరియు మే dashboard lo ఏమీ కనిపించదు.

6. Cache ఎంత బాగా పని చేస్తే stampede అంత పెద్దది. Stampede పరిమాణం = hit rate × DB latency. అందుకే విజయమే ఇక్కడ ప్రమాదాన్ని తెస్తుంది – 400 queries ఒకే key కోసం.

7. Test విఫలమైనప్పుడు మొదట model ని అనుమానించండి. నా fuzz 800 lo 624 సార్లు విఫలమైంది – ఎందుకంటే నేను "4 servers" అనుకున్నది నిజానికి **4 వేరే systems**. అది సరిచేయడం ఒక design నిజాన్ని బయటపెట్టింది: **generated keys కి coordination అవసరం లేదు, aliases కి అవసరం.**

8. కొన్నిసార్లు bug ని కనిపించేలా చేయడానికి ముందు model ని నిజాయితీగా చేయాలి. నా cache ఒక reference నిల్వ చేసేది, Redis లాగా ఒక ప్రతికాదు. అందువల్ల invalidate() పంక్తి అనవసరం అని కనిపించేది. Model ని సరిచేశాక ఆ పంక్తి తీసేయడం **300 lo 257 సార్లు** విఫలమైంది.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ problem lo ప్రతి కష్టమైన నిర్ణయం "ఈ key ఎవరిది?" అనే ఒక్క ప్రశ్న చుట్టూ తిరుగుతుంది.

Key ఎవరైనా ఊహించగలిగితే అది అందరిదీ (§4). Browser దాన్ని cache చేసుకుంటే అది *browser* ది, మీది కాదు (§7). Cache అందరికీ ఒకేసారి miss అయితే అది *database* ది (§10).

మూడు విరుపులూ ఒకే ఆకారంలో ఉన్నాయి: **మీరు మీది అనుకున్న ఒక వస్తువు, నిజానికి మీ అదుపులో లేదు.**